Passo a passo: medir o uso de memória RAM dos jogos

Este guia explica como medir, em jogos leves, médios e pesados, a **memória virtual alocada**, o **pico de memória alocada** e a **memória residente em RAM**, usando Python no Windows.

O que cada métrica significa

- **Memória virtual alocada:** tudo o que o jogo reservou de memória (commit size / Private Bytes). Nem tudo está ocupando a RAM de verdade.
- **Pico de memória alocada:** o maior valor de memória virtual que o jogo chegou a alocar durante a execução.
- **Memória residente em RAM:** a parte que de fato está ocupando a RAM naquele momento (Working Set).

O script também registra o pico da memória residente, que é um dado útil para comparar computadores.

Parte 1: Preparar o computador (só uma vez por PC)

1. Baixe e instale o Python em **python.org/downloads**. Na primeira tela do instalador, marque a caixa **"Add python.exe to PATH"**. Esse passo é o que mais costuma ser esquecido.
2. Depois de instalar, **feche e abra de novo** o PowerShell (ou Prompt de Comando). Para abrir: aperte a tecla Windows, digite `powershell` e aperte Enter.
3. Confirme que o Python foi instalado:

```
python --version
```

4. Instale a biblioteca psutil:

```
pip install psutil
```

Se o `pip` não for reconhecido, use:

```
python -m pip install psutil
```

Quando terminar, deve aparecer `Successfully installed psutil`.

5. Crie uma pasta, por exemplo `C:\teste`, abra o **Bloco de Notas**, cole o script da seção abaixo e salve como `medir.py` dentro dela. Em "Tipo", escolha **Todos os arquivos**, senão ele será salvo como `.txt`.

O script (medir.py)

```
import psutil, time, csv, platform, argparse

IS_WIN = platform.system() == "Windows"
MB = 1024 * 1024

def find_procs(name):
    return [p for p in psutil.process_iter(["name"])
            if p.info["name"] and p.info["name"].lower() == name.lower()]

def read_proc(p):
    """Retorna (virtual, resident, pico_virtual, pico_resident) em bytes."""
    mi = p.memory_info()
    if IS_WIN:
        return mi.vms, mi.rss, mi.peak_pagefile, mi.peak_wset
    # Linux: lê os picos do /proc
    peak_v = peak_r = 0
    with open(f"/proc/{p.pid}/status") as f:
        for line in f:
            if line.startswith("VmPeak:"):
                peak_v = int(line.split()[1]) * 1024
            elif line.startswith("VmHWM:"):
                peak_r = int(line.split()[1]) * 1024
    return mi.vms, mi.rss, peak_v, peak_r

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("process", help="nome do executável, ex: game.exe")
    ap.add_argument("--duration", type=int, default=600, help="segundos")
    ap.add_argument("--interval", type=float, default=1.0)
    ap.add_argument("--out", default="medicao.csv")
    a = ap.parse_args()

    procs = find_procs(a.process)
    if not procs:
        raise SystemExit(f"Processo '{a.process}' não encontrado.")

    max_virt = max_res = 0
    peak_virt_os = peak_res_os = 0
    t0 = time.time()
```

```
    with open(a.out, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["t_s", "virtual_MB", "residente_MB",
                    "pico_virtual_SO_MB", "pico_residente_SO_MB"])
        while time.time() - t0 < a.duration:
            v = r = pv = pr = 0
            for p in procs:
                try:
                    a_v, a_r, a_pv, a_pr = read_proc(p)
                    v += a_v; r += a_r; pv += a_pv; pr += a_pr
                except (psutil.NoSuchProcess, psutil.AccessDenied):
                    pass
            max_virt, max_res = max(max_virt, v), max(max_res, r)
            peak_virt_os, peak_res_os = max(peak_virt_os, pv),
max(peak_res_os, pr)
            w.writerow([round(time.time() - t0, 1), v // MB, r // MB,
                        pv // MB, pr // MB])
            f.flush()
            time.sleep(a.interval)

    print(f"Pico virtual (amostrado):   {max_virt // MB} MB")
    print(f"Pico residente (amostrado): {max_res // MB} MB")
    print(f"Pico virtual (reportado pelo SO):   {peak_virt_os // MB} MB")
    print(f"Pico residente (reportado pelo SO): {peak_res_os // MB} MB")

if __name__ == "__main__":
    main()
```

Parte 2: Medir um jogo

1. **Reinicie o PC** e feche programas em segundo plano (navegador, Discord, etc.). Anote quanta RAM está livre no início.
2. **Abra o jogo** e deixe-o no ponto de teste (por exemplo, já dentro da fase escolhida).
3. Descubra o **nome do executável**: aperte `Ctrl + Shift + Esc` para abrir o Gerenciador de Tarefas, localize o jogo, clique com o botão direito e escolha **Ir para detalhes**. O nome com `.exe` aparecerá destacado (ex.: `eldenring.exe`).
4. No PowerShell, entre na pasta do script:

```
cd C:\teste
```

5. Rode a medição (600 segundos = 10 minutos):

```
python medir.py eldenring.exe --duration 600 --out eldenring_pc1_teste1.csv
```

6. **Jogue normalmente** durante o tempo da medição. Você pode alternar para outras janelas com Alt+Tab, mas deixe o PowerShell aberto.
7. Ao final, o script mostra na tela os picos e salva o arquivo `.csv` na pasta `C:\teste`.

Como parar a medição

- **Esperar acabar:** o script termina sozinho quando o tempo definido chega ao fim e mostra o resumo na tela.
- **Parar antes:** clique na janela do PowerShell e aperte **Ctrl + C**. O resumo final não aparece, mas o `.csv` já foi gravado a cada segundo. Abra no Excel e use a função MÁXIMO nas colunas de pico.

O que tem no arquivo .csv

Coluna	Significado
`t_s`	Tempo em segundos desde o início da medição
`virtual_MB`	Memória virtual alocada naquele instante
`residente_MB`	Memória residente em RAM naquele instante
`pico_virtual_SO_MB`	Pico de memória alocada reportado pelo Windows
`pico_residente_SO_MB`	Pico de memória residente reportado pelo Windows

Parte 3: Repetir para todo o projeto

1. **Classifique os jogos** antes de começar, com um critério claro (por exemplo, requisitos mínimos oficiais, tamanho em disco ou tipo de gráfico). Escolha de 2 a 3 jogos por categoria: leve, médio e pesado.
2. **Padronize o cenário:** use o mesmo trecho do jogo (mesma fase, mesmo tempo), a mesma resolução e as mesmas configurações gráficas em todos os computadores.
3. **Repita 3 vezes** cada teste e calcule média e desvio padrão.
4. **Nomeie os arquivos** de forma organizada, por exemplo `jogo_pc1_teste1.csv`, `jogo_pc1_teste2.csv`, `jogo_pc2_teste1.csv`. Assim nada se mistura.

5. **Registre a ficha de cada computador:** processador, quantidade e velocidade da RAM, placa de vídeo e VRAM, sistema operacional e se o jogo está em SSD ou HDD.

Tabela sugerida para organizar os resultados

Jogo	Categoria	PC	Teste	Virtual máx (MB)	Pico alocado (MB)	Residente máx (MB)
	Leve	PC1	1			
	Leve	PC1	2			
	Leve	PC1	3			

Observações importantes

- O script soma todos os processos com o mesmo nome. Se o jogo tiver launcher ou processos auxiliares, confira no Gerenciador de Tarefas qual é o executável principal.
- O pico "amostrado" depende do intervalo de medição (1 segundo por padrão). O pico reportado pelo Windows costuma ser mais fiel.
- Se o jogo for fechado antes do fim da medição, o script pode parar ou registrar zeros. Nesse caso, rode novamente com o jogo aberto.
- Algumas medições podem pedir que o PowerShell seja aberto como administrador, principalmente em jogos com anti-cheat.

Problemas comuns

Erro	Solução
`pip` não é reconhecido	Reinstale o Python marcando "Add python.exe to PATH", reabra o PowerShell, ou use `python -m pip install psutil`
`Processo ... não encontrado`	Confira o nome exato do `.exe` na aba Detalhes do Gerenciador de Tarefas e deixe o jogo aberto antes de rodar
Arquivo salvo como `medir.py.txt`	No Bloco de Notas, escolha "Todos os arquivos" em Tipo ao salvar
Acesso negado	Abra o PowerShell como administrador